Few-shot Task Learning via Compositional Concept Inference

Hanming Ye **Yiding Song** **Yilun Du**
Harvard University

Abstract

Humans can learn new tasks from just a few demonstrations. Prior methods like behavior cloning struggle to replicate this ability because they imitate demonstrations without learning the underlying concepts, such as goals, positional relations, and constraints. To address this challenge, we propose COIN, an approach that solves novel tasks by inferring familiar concepts from contextual demonstrations then composing them. We implement this approach with a diffusion model, and show that it is effective at few-shot learning across diverse domains. On tasks including object rearrangement and goal-oriented navigation, COIN recombines previously seen concepts to solve novel tasks at inference time. On the LIBERO robotics benchmark, COIN achieves a 96.6% average success rate on LIBERO-90, matching pretrained policies such as π_0 with roughly $7\times$ fewer parameters. When fine-tuning on unseen tasks with limited demonstrations, COIN outperforms them, highlighting the advantage of concept inference over end-to-end policy adaptation in the few-shot regime.

1 Introduction

Humans can learn a new task by observing a few demonstrations. We do not simply copy the demonstrated motions, but also infer the concepts behind them and recombine them in new situations. After watching someone shelve books by title, we can shelve a new set of books on a different shelf, even though no individual motion repeats. This ability to extract concepts from a few examples and then compose these concepts in novel situations is a defining feature of human task learning.

Replicating this ability in learned policies remains difficult. Behavior cloning [32] imitates demonstrated actions directly, so it does not explain how to reuse the structure that tasks share. Consequently, fine-tuning on a few demonstrations of a new task is brittle, since the policy needed for the new task may lie far from the policies encountered during training. Language conditioning offers a route for specifying task structure, and recent vision-language-action (VLA) models have shown that large pretrained representations can support broad robotic generalization [3, 45, 23, 30, 2]. However, language is not always a sufficient representation of a task. The same instruction can require different behaviors depending on object geometry or scene layout, and language descriptions often omit these procedural details [7].

In this work, we introduce COIN, a framework for few-shot imitation learning through compositional inference (Figure 1). It is motivated by two core ideas:

- **The behavior required for a new task may be globally unfamiliar, but can be solved by decomposing it into locally familiar concepts** [9]. For example, a model that has learned the relation "inside" and the attribute "red" can compose them to put the red object inside the box, even if it only saw these concepts in separate tasks. To leverage this idea, COIN represents behavior in terms of *task concepts*: reusable components of behavior such as spatial relations, motion styles, and constraints. During execution, we first infer the concepts that describe context demonstrations, then compose these concepts into motion using a policy. In practice, we use a diffusion model to predict one noise term per concept and sum these predictions. We evaluate COIN's ability

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

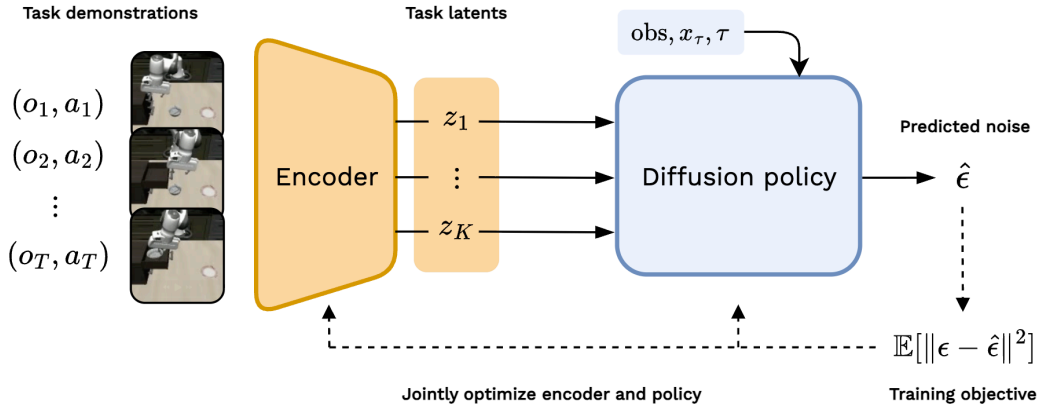


Figure 1: **COIN overview.** We propose a method to learn tasks few-shot via "compositional concept inference". We infer concepts that describe a new task from contextual demonstrations, then compose these concepts into an action prediction. In practice, an encoder maps context demonstrations into K latent concepts. A policy predicts one noise term per latent and sums them. The encoder and policy are trained jointly with a single denoising loss on a target trajectory sampled independently of the context demonstrations. Consequently, the encoder learns to represent the context as concepts that are useful for action generation.

for extracting and combining concepts across object rearrangement and goal-oriented navigation, where COIN solves held-out tasks more reliably than baselines.

- **Task representations can be taught by training for control.** Previous work has encoded language descriptions of tasks or used metric-learning [20] to construct task embeddings that condition policies. However, recent works suggest that these representations may not be sufficiently rich for policies to generalize to unseen tasks [7]. To build task concepts that carry the information needed to reproduce complex behavior, we train the encoder on the same signal as the policy. In particular, we jointly optimize the encoder and the diffusion policy on the same denoising loss. We claim that this helps the model select the intended behavior among multiple plausible modes, which is crucial when tasks share similar goals or language instructions but differ in scene geometry or object configuration. In LIBERO, where many manipulation tasks have similar prompts but require different behaviors across scenes, COIN consistently outperforms strong baselines in few-shot experiments. Surprisingly, even when all data is made available, **COIN matches VLAs like π_0 with roughly $7\times$ fewer parameters and achieves an average success rate of 96.6% on LIBERO-90.** We further consolidate our claim by showing that conditioning on COIN's task concepts is more effective than conditioning on alternatives like language or metric-learned embeddings.

Together, inferring then composing task concepts trained on action prediction allows COIN to learn new tasks more effectively from limited demonstrations. Our contributions are: **(1)** We propose COIN, a framework for few-shot imitation learning that infers reusable concepts from task demonstrations and composes them to specify new behaviors. **(2)** We evaluate COIN across object rearrangement and goal-oriented navigation, showing that concept composition enables stronger few-shot inference on held-out task compositions. We further evaluate COIN on LIBERO robotics tasks, showing that concepts inferred from demonstrations substantially improve few-shot fine-tuning over prior state-of-the-art policies. **(3)** We analyze the mechanisms behind COIN's performance, providing evidence that inferring task concepts and demonstration-conditioned mode selection help the policy learn new tasks from few demonstrations.

2 Related Work

Few-shot imitation learning. Imitation learning traditionally trains policies to match demonstrated actions through behavior cloning [32], or infers rewards from demonstrations through inverse reinforcement learning [29, 44, 18]. While effective with sufficient data, these approaches do not directly address how an agent should infer and reuse task structure from only a few examples. Meta-

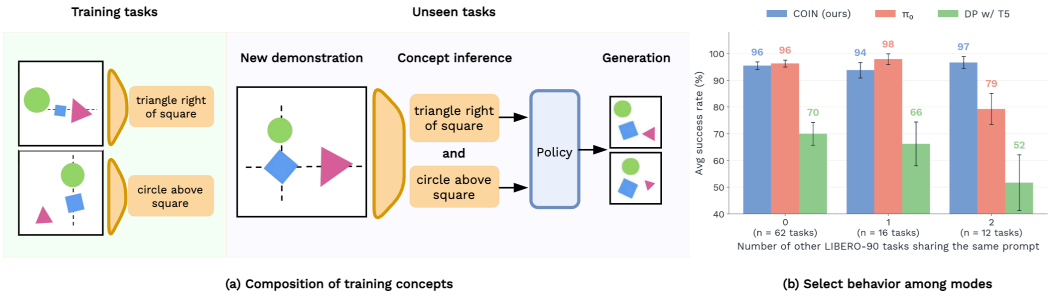


Figure 2: **Mechanisms of COIN. (a)** COIN infers reusable concepts from demonstrations and composes them to solve unseen tasks, such as combining separately learned spatial relations into a new object arrangement. **(b)** Demonstration-inferred concepts help select the intended behavior when multiple tasks share similar or identical language prompts.

imitation and one-shot imitation methods instead train policies to adapt from a demonstration of a new task [12, 14, 43]. More recent in-context imitation approaches condition policies on demonstration trajectories at test time, treating demonstrations as prompts for action prediction [40, 36, 41, 6]. TecNets [20] instead compress demonstrations into a task embedding trained with a metric-learning loss. COIN differs from TecNets in two ways: its latents receive only the policy's denoising loss, and a demonstration maps to several latents whose noise predictions compose. These methods show that demonstrations can serve as rich task specifications.

Inverse generative modeling. A close prior work is FTL-IGM [28], which casts few-shot task learning as inverse generative modeling. It trains a generative model conditioned on language (T5) embeddings of the training concepts, and learns a new task by optimizing a latent so that the frozen model assigns high likelihood to the demonstrations. COIN differs in both halves: its task representation is learned from demonstrations through the policy's own loss rather than taken from a language model, and a new task is inferred by a forward pass rather than by test-time optimization. Sections 5.1 and 5.2 measure both differences.

Generative and language-conditioned robot policies. Diffusion policies model multimodal action distributions by iterative denoising [19, 21, 1, 4], and MoDE routes denoising through a mixture of experts [33]. Vision-language-action models such as RT-1, RT-2, OpenVLA, Octo, π_0, and $\pi_{0.5}$ condition on language and inherit web-scale vision-language pretraining [3, 45, 23, 30, 2, 31], and are adapted to new tasks by fine-tuning [22]. Language is a compact and interpretable task specification, whereas demonstrations carry procedural detail at a higher collection cost [5]. COIN conditions on demonstrations and uses no language.

Compositional generative models. Energy-based and diffusion models can represent concepts as separate energies or scores that are summed at inference time [10, 25, 11], and Decomp Diffusion discovers such components without supervision [35]. These methods infer components from the same image they reconstruct. COIN infers them from a different execution of the task and trains them through action generation. In robotics, composition has been used to combine diffusion policies, constraints, and guidance signals [21, 37, 38]. COIN uses learned, composable concepts for few-shot imitation.

3 Compositional Concept Inference

COIN learns new tasks by inferring latent task concepts from demonstrations and using those concepts to condition a diffusion policy. The method has two components: an encoder E_ϕ that maps a demonstration into a set of concept latents, and a denoiser ϵ_θ that predicts actions conditioned on one concept latent at a time. We first formalize few-shot task learning and motivate representing tasks compositionally (Section 3.1). Then, we find that turning this idea into a few-shot learner raises three challenges: the concepts of a new task are not labeled, inferring them invites a shortcut, and the inferred concepts must combine during generation. In turn, we address these in Sections 3.2 and 3.3. Finally, Section 3.4 describes how COIN learns and executes a new task.

3.1 Few-shot task learning

We assume a multi-task demonstration dataset. Each task m contains demonstrations $\mathcal{D}_m = \{\tau_m^i\}_{i=1}^{M_m}$, where each demonstration $\tau = (o_{1:H}, a_{1:H})$ consists of observations and actions. Training covers a set of tasks $\mathcal{M}_{\text{train}}$. At test time, a new task $m^* \notin \mathcal{M}_{\text{train}}$ is specified by k *context demonstrations* $\mathcal{C} = \{\tau^1, \ldots, \tau^k\} \subset \mathcal{D}_{m^*}$: examples that indicate what behavior should be performed. The goal is a policy $\pi(a \mid o, \mathcal{C})$ that performs task m^* from new initial states.

Each task is composed of concepts such as attributes and relations. If each task is defined by a set of concepts $c_{1:K}$ drawn from a shared vocabulary, the number of possible tasks grows combinatorially with the vocabulary, while any training set covers only a small fraction of the combinations. A new task may therefore be globally unfamiliar (e.g., as measured in trajectory space), yet composed of locally familiar concepts.

A natural way to state that a behavior must satisfy every concept of its task is a product of experts [10, 25]:

$$p(x \mid c_{1:K}) \;\propto\; p(x) \prod_{k=1}^{K} \frac{p(x \mid c_k)}{p(x)}, \tag{1}$$

under which a behavior x is likely only if it is likely under each concept. The score of this product is a sum of per-concept scores,

$$\nabla_x \log p(x \mid c_{1:K}) \;=\; \sum_{k=1}^{K} \nabla_x \log p(x \mid c_k) \;-\; (K-1)\, \nabla_x \log p(x), \tag{2}$$

and a diffusion model's noise prediction is a scaled score, $\epsilon(x_t, t) = -\sqrt{1-\bar{\alpha}_t}\, \nabla_{x_t} \log p_t(x_t)$. Composing concepts therefore amounts to adding per-concept noise predictions [25]. A model that has learned each term can generate a combination of concepts it has never seen together, because each term is evaluated on a concept it was trained on. Equation (2) motivates COIN's architecture (though it does not make the summed prediction an exact sampler for the product).

3.2 Inferring concepts from demonstrations

Given that we want to model behavior as a composition of concepts, the subsequent challenge is that **concepts are not labeled**. Demonstrations do not come annotated with the concepts they instantiate, and for most tasks no vocabulary of concepts is given at all. Language provides a partial vocabulary, but the same instruction can require different behaviors in different scenes, and instructions omit procedural detail [7]. Embeddings trained with an objective of their own, such as the metric-learning loss of TecNets [20], organize demonstrations by task identity, which is not necessarily what the policy needs in order to act.

To solve this, we use **amortized inference of concepts trained for control**. Given a context demonstration τ, COIN encodes it into K concept latents:

$$E_\phi(\tau) = \{z_1, \ldots, z_K\}, \qquad z_k \in \mathbb{R}^L. \tag{3}$$

The latents are learned end-to-end through the action-generation objective. We expect them to capture any reusable behavioral factor useful for control, such as goals, spatial relations, object configurations, or constraints. The encoder only receives the gradient of the policy's denoising loss as its training signal (Equation (6)). At test time, the concepts of a new task are inferred in a single forward pass, rather than from optimizing a latent against a frozen generative model, $z^* = \arg\min_z \mathcal{L}(z)$, as in inverse generative modeling [28].

In practice, we use an MLP as the encoder. For low-dimensional domains, the MLP operates on pooled trajectory features. For image-based manipulation, demonstration frames are first embedded with the same observation encoder used by the policy, and the resulting features are mapped to concept latents.

Another challenge is that **the encoder can take a shortcut** by lowering the action-prediction loss without learning the task: if it sees the trajectory the policy must denoise, it can simply describe that trajectory. However, we want the latents to encode concepts that represent the overall behavior for the task, not any particular trajectory. To solve this, during training, we randomly sample a task then sample the context demonstration and the target trajectory independently from that task:

$$m \sim p(m), \qquad \tau_c \sim \mathcal{D}_m, \qquad \tau \sim \mathcal{D}_m. \tag{4}$$

Algorithm 1 Training COIN

Require: datasets $\{\mathcal{D}_m\}$; E_ϕ, ϵ_θ

repeat
  $m \sim p(m)$ ▷ sample a task
  $\tau_c \sim \mathcal{D}_m,\ \tau \sim \mathcal{D}_m$ ▷ independently
  $\{z_1, \dots, z_K\} \leftarrow E_\phi(\tau_c)$ ▷ infer concepts
  $(o, x_0) \leftarrow$ observation and actions from τ
  $t \sim \mathcal{U}\{1, \dots, T\},\ \epsilon \sim \mathcal{N}(0, I)$
  $x_t \leftarrow \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$
  $\hat{\epsilon} \leftarrow \sum_k \epsilon_\theta(x_t, t, o, z_k)$ ▷ compose
  update θ, ϕ by $\nabla_{\theta,\phi} \|\epsilon - \hat{\epsilon}\|_2^2$
until converged

Algorithm 2 Learning and executing a new task

Require: demonstrations $\mathcal{C}$; trained E_ϕ, ϵ_θ

if fine-tuning **then**
  run Algorithm 1 on $\mathcal{C}$
end if
$Z \leftarrow \bigcup_{i=1}^{k} E_\phi(\tau^i)$ ▷ infer concepts once
while task not done **do**
  observe o; $x_T \sim \mathcal{N}(0, I)$
  for $t = T, \dots, 1$ **do**
    $\hat{\epsilon} \leftarrow \sum_{z \in Z} \epsilon_\theta(x_t, t, o, z)$ ▷ compose
    $x_{t-1} \leftarrow \textsc{SamplerStep}(x_t, \hat{\epsilon}, t)$
  end for
  execute the first actions of x_0
end while

Thus, *the encoder is trained to infer information from one rollout that helps predict actions in another rollout of the same task.* As the ablations in Table 2 show, this improves success substantially.

3.3 Composing concepts in the denoiser

After inferring concepts from demonstrations, we need to compose these concepts during generation. If we use a single conditioning vector (e.g., the average of the concept latents), then this vector could be outside the training distribution seen by the policy. COIN instead keeps the concepts separate until generation and combines them in the noise prediction.

Additive composition. COIN uses a diffusion policy to model the action distribution [19, 4]. Let x_0 denote the target action, action chunk, or trajectory segment. We sample a diffusion step t and noise $\epsilon \sim \mathcal{N}(0, I)$, and form $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$. A denoising network $\epsilon_\theta(x_t, t, o, z_k)$ predicts the added noise from the noisy action variable, diffusion step, current observation, and one concept latent. The key operation is composition across concept latents. For a demonstration encoded as $\{z_1, \dots, z_K\}$, COIN predicts

$$\hat{\epsilon}_\theta(x_t, t, o, \tau_c) = \sum_{k=1}^{K} \epsilon_\theta(x_t, t, o, z_k). \tag{5}$$

The encoder and policy are trained jointly with the composed denoising loss, and with nothing else:

$$\mathcal{L}_{\text{COIN}}(\theta, \phi) = \mathbb{E}_{m,\, \tau_c,\, \tau,\, t,\, \epsilon} \left[\left\| \epsilon - \sum_{k=1}^{K} \epsilon_\theta(x_t, t, o, z_k) \right\|_2^2 \right], \qquad \{z_k\}_{k=1}^{K} = E_\phi(\tau_c), \tag{6}$$

where m, τ_c, and τ follow Equation (4), and the observation o and target x_0 come from τ (Algorithm 1). Here, each latent contributes an additive denoising direction, and the policy learns how these directions combine to explain the target behavior. This design is motivated by score-based composition in diffusion models [25, 37], but the concept latents and denoiser are trained jointly for imitation learning.

When multiple context demonstrations are provided, COIN encodes them independently and concatenates their concept latents. The denoiser then receives each latent through the same shared composition rule. Because the rule is the same for every latent, latents can also be pooled across demonstrations of different concepts (Section 4.2).

3.4 Learning and executing a new task

To learn a new task, COIN encodes its context demonstrations once, $Z = \bigcup_i E_\phi(\tau^i)$, and holds the resulting concept latents fixed while sampling actions with the composed prediction of Equation (5) (Algorithm 2). In the concept-learning domains (Section 4) this is all COIN does and no weights are updated. On held-out LIBERO tasks (Section 5.2), we additionally fine-tune encoder and policy on the same k demonstrations with $\mathcal{L}_{\text{COIN}}$, and at evaluation the encoder sees only those k demonstrations.

COIN wraps around any conditional diffusion or flow-matching model, so the same concept-composition rule can be used with different action-generation architectures. Object rearrangement uses an MLP denoiser for single-step prediction. Goal navigation uses a temporal U-Net over trajectory segments. In these models, the diffusion timestep, current conditioning observation, and concept

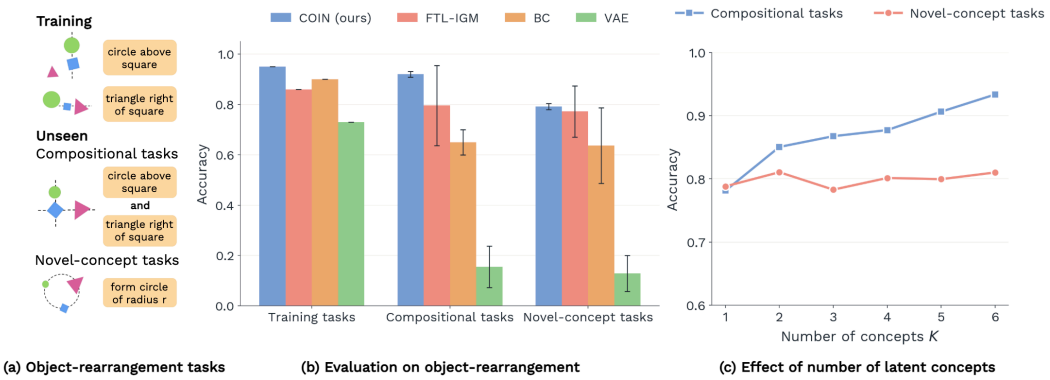


Figure 3: **Object rearrangement. (a)** Training tasks contain a single spatial relation between two objects, while held-out compositional tasks require satisfying multiple training relations simultaneously and novel-concept tasks require relations not explicitly seen during training. **(b)** COIN outperforms baselines on all categories of tasks. **(c)** Increasing the number of concept latents K improves accuracy on compositions of training concepts but has little effect on novel-concept tasks.

latent are concatenated into a conditioning vector that is injected into the denoising network. For LIBERO, we use an image-based transformer diffusion policy over action chunks [4].

4 Composing Concepts to Learn New Tasks

We first test whether COIN learns new tasks by inference alone, in two domains adapted from Netanyahu et al. [28]: object rearrangement and goal-oriented navigation. Evaluation covers training concepts, compositions of training concepts that training never showed together, and, in object rearrangement, novel concepts not seen in training. The model receives five demonstrations of a held-out task, infers its concept latents, and generates behavior without any weight update.

Baselines. FTL-IGM optimizes latent variables under a frozen conditional diffusion model [28]. **BC** is a goal-conditioned behavior-cloning policy whose conditioning input is optimized from the demonstrations. **VAE** learns a reconstructive latent space and optimizes a latent for each new task. In navigation we also compare to an **in-context** policy that conditions directly on demonstration trajectories.

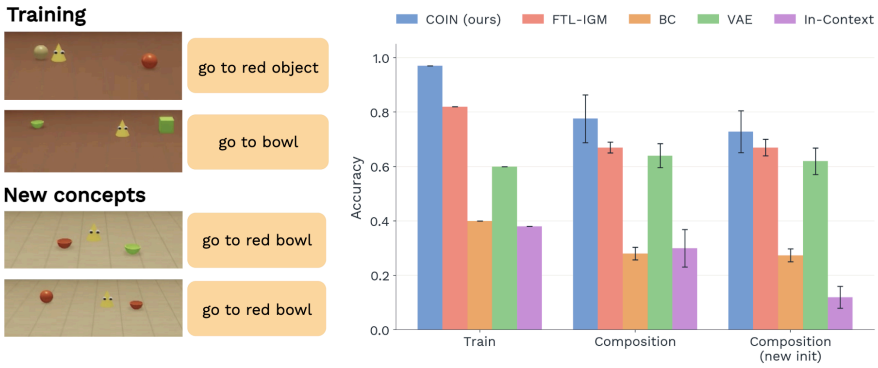
4.1 Object rearrangement: composing spatial relations

The object-rearrangement domain tests whether a model can infer relational concepts from demonstrations and combine them into new constraints. Each scene contains three geometric objects with continuous positions. Training concepts specify a single pairwise relation, such as placing one object above or to the right of another. At evaluation, the model must either reproduce training relations, compose two training relations in a new conjunction, or infer a novel relation such as a diagonal or circular arrangement.

Figure 3 shows that COIN performs strongly across all three settings. This directly tests compositionality, as the model must satisfy multiple relational constraints simultaneously, even though each constraint was learned separately. BC and VAE degrade substantially in the compositional regime, suggesting that direct conditioning and reconstructive latents do not reliably capture reusable relational structure. FTL-IGM remains competitive, but COIN achieves stronger compositional performance while using amortized concept inference rather than test-time latent optimization. The strong performance of COIN, particularly on compositional tasks, suggests that the concept inference approach indeed combines separately learned relational factors into a coherent generation.

4.2 Effect of the number of concept slots

We next vary the number of latent concepts K to test whether COIN's gains are tied to its compositional representation. Figure 3c shows a clear separation between two regimes. Accuracy on compositions of training concepts improves steadily as K increases, while performance on novel-concept tasks remains comparatively flat.



(a) Goal-oriented navigation tasks **(b) Evaluation on training and new tasks**

Figure 4: **Goal-oriented navigation.** **(a)** Training tasks specify a single attribute, while held-out tasks require two attributes, such as navigating to the red bowl, and distractors share one attribute with the target. **(b)** COIN achieves the highest accuracy on training tasks, held-out compositions, and compositions evaluated from new initial states.

This pattern suggests that more latent concepts help represent and recombine structure already covered by the training concept family, but do not by themselves solve extrapolation to concepts outside that family. In other words, increasing K improves the resolution of the learned compositional space, but does not necessarily expand its support. This pattern supports the interpretation that COIN's main advantage in the previous experiments comes from compositional recombination, while novel concepts remain limited by the distance between the new concept and the training distribution.

To further isolate the effect of COIN's concept composition method, we fix the trained models and vary only how two concepts are combined on held-out compositions. Averaging the two latents and denoising once succeeds in 0.33 of cases, averaging the two noise predictions in 0.70, and keeping each relation in its own latent in 0.98. Combining the noise predictions thus more than doubles success over averaging the latents, and keeping each relation in its own latent lifts success to near-perfect. This last step is possible only because the encoder has separated the two relations into different latents: had it not, rule (iii) would add nothing over rule (ii) (details in Appendix A.1).

4.3 Goal-oriented navigation

The goal-navigation domain tests whether concept inference can identify the intended target under distractors. An agent must navigate to one of two objects, each defined by color and shape. Training tasks specify a single target attribute, such as "go to the red object" or "go to the bowl." At test time, tasks require a novel binding of two attributes, such as "go to the red bowl." Distractors share one attribute with the target, so success requires binding color and shape rather than following either attribute alone.

Figure 4 shows that COIN outperforms all baselines on training tasks, held-out attribute compositions, and compositions evaluated from new initial states. The new-initial-state setting is especially diagnostic because the model cannot succeed by reproducing the geometry of the demonstrations. It must infer which object is the target and navigate to it from a different starting configuration. COIN's performance in this setting suggests that the inferred concepts encode task intent, such as the target attribute binding, rather than only trajectory-level correlations.

5 Few-Shot Learning on LIBERO Manipulation Tasks

We next evaluate COIN on robot manipulation in LIBERO [24]. These experiments ask three questions: whether demonstration-inferred concepts scale to high-dimensional control, whether they improve few-shot adaptation to manipulation tasks, and which parts of the model are responsible for this improvement.

We consider two evaluation settings. First, we train and evaluate on LIBERO-90. Second, following prior few-shot manipulation work [42, 15], we pretrain each model on 80 tasks from LIBERO-90 and fine-tune on the remaining 10 tasks with a limited number of demonstrations per task (Appendix D). We report average success rate over 20 rollouts per task unless stated otherwise.

Method	LIBERO-90
Diffusion Policy	64.5
BAKU [16]	90.0
MoDE [33]	91.0
π_0 [2]	92.3
FLOWER [34]	94.7
COIN	$\mathbf{96.6 \pm 0.2}$

Table 1: **LIBERO-90** comparison with prior methods.

Variant	Success
conditioning: task label	66.2
conditioning: frozen T5	64.5
conditioning: trainable T5	84.3
context: fixed pair	80.7
context: target trajectory	63.5
encoder loss: TecNets	86.4
encoder loss: variational	85.0
COIN	**95.1**

Table 2: **Removing one component** of COIN.

Method	1 demo	5 demos
language (trainable T5)	39.8	67.2 ± 0.4
in-context	38.5	70.5
π_0	36.5	59.5
TecNets	33.6 ± 0.4	64.3
task label	25.3	65.5 ± 1.4
COIN	$\mathbf{52.3 \pm 4.4}$	$\mathbf{77.5 \pm 0.3}$

Table 3: **Few-shot learning** of held-out tasks.

Baselines. To isolate the task representation, most baselines share COIN's policy exactly (a 24-layer transformer denoiser with ResNet visual encoders, the same data processing and training recipe) and differ only in the conditioning module. **Task label** conditions on a one-hot task identity. **Language** conditions on a T5-base embedding of the instruction, frozen or fine-tuned, placed where COIN's task latent goes. **TecNets-style** trains COIN's encoder with the TecNets recipe [20]: an L2-normalized embedding with a hinge-margin metric loss, averaged over demonstrations. **Variational** adds a KL penalty (10^{-4}) on a Gaussian encoder to the denoising loss and uses the mean at evaluation. **In-context** has no encoder: the policy's observation encoder embeds the demonstration frames, which the denoiser attends to as context tokens. **Test-time optimization** follows FTL-IGM's mechanism: it optimizes a task latent against the frozen pretrained policy. Apart from the trainable T5 encoder, every baseline is within 3% of COIN's parameter count (Appendix B.1). We also fine-tune π_0 [2] ourselves with its official recipe and evaluate it under our protocol.

On LIBERO, we find that COIN with a single pooled latent ($K{=}1$) performs on par with additive composition of $K{=}2$ latents at matched capacity (95.1 vs. 94.4 on LIBERO-90). We hypothesize that this is because LIBERO data and tasks are not sufficiently compositional. Most LIBERO-90 tasks have a single goal (79 of 90 have one goal condition; Appendix B.2). Consequently, without pretraining on additional data, extra concept latents likely bring no gain. We therefore use $K{=}1$ for all LIBERO results.

5.1 Evaluation on LIBERO-90

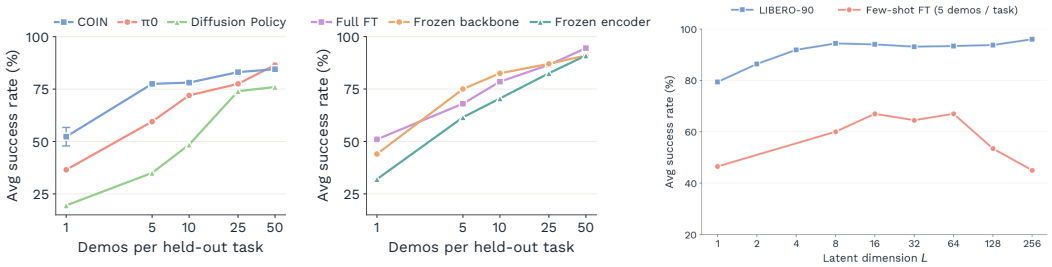
COIN outperforms the published results in Table 1 (Table 7 lists all published LIBERO-90 results), and on the same 80 tasks it ties π_0, which we fine-tune with its official recipe and evaluate under our protocol. This result is notable because COIN does not rely on language instructions, web-scale vision-language pretraining, or a large VLA backbone. Task information enters only through latents inferred from demonstrations. COIN ties π_0 with roughly $7\times$ fewer parameters (0.47B vs. 3.3B), suggesting that concept inference from demonstrations can be an efficient alternative to scaling language-based backbones.

We examine why COIN is effective at robot manipulation, and find that three components are necessary: conditioning on demonstrations, sampling the context demonstration independently of the target trajectory, and training the encoder through the policy's own loss. Removing or replacing any one of them at matched architecture and data lowers success (Table 2). This empirically illustrates how COIN's differences from previous methods enable its generalization. Beyond direct latent inference from demonstrations, COIN also trains task representations for control, through the policy's own loss. The TecNets-style and variational encoders are explicit task embeddings as well, so the gain comes from the training signal rather than from having an embedding at all.

5.2 Few-shot fine-tuning on held-out tasks

We next evaluate adaptation to held-out LIBERO tasks. Models are pretrained on LIBERO-80 and fine-tuned on the held-out 10 tasks with 1, 5, 10, 25, or 50 demonstrations per task. Figure 5a shows that COIN outperforms Diffusion Policy at every data budget and π_0 at every budget up to 25 demonstrations. The advantage is largest when demonstrations are scarce: with only one demonstration per held-out task, COIN reaches roughly 52% average success, compared to roughly 37% for π_0 and 20% for Diffusion Policy.

As the number of demonstrations increases, all methods improve, and COIN keeps its advantage up to 25 demonstrations. This suggests that COIN is not only learning a stronger in-distribution



(a) Few-shot fine-tuning **(b)** Freezing parts of COIN **(c)** Concept latent dimension

Figure 5: **Few-shot fine-tuning and latent dimension on LIBERO. (a)** Models are pretrained on LIBERO-80 and fine-tuned on held-out tasks with a limited number of demonstrations per task. COIN outperforms Diffusion Policy at every budget and π_0 up to 25 demonstrations, with the relative improvement increasing as data becomes more limited. **(b)** Freezing parts of COIN during fine-tuning. Freezing the denoising backbone performs similarly to full fine-tuning, while freezing the demonstration encoder hurts performance in low-data regimes. **(c)** Intermediate latent sizes perform best and very small latents underfit, while very large latents hurt few-shot adaptation.

manipulation policy, but also a representation that transfers more efficiently to new tasks. Rather than adapting from only a language instruction or a monolithic policy state, COIN adapts through a demonstration encoder trained to map examples into behaviorally useful concept latents.

One explanation is that COIN forms more expressive task latents. When several tasks share an instruction, language cannot say which one is meant, but a demonstration can. We group LIBERO-90 tasks by the number of other tasks that share the exact same language prompt. Figure 2b shows that COIN remains stable as prompt sharing increases. On tasks with no shared prompt, COIN and π_0 both achieve 96% success. When two other tasks share the same prompt, however, π_0 drops to 79%, and a language-conditioned Diffusion Policy baseline drops to 52%. COIN remains at 97%. This trend is consistent with the argument that when language is less discriminative, demonstrations provide additional procedural evidence about which behavior is intended. Moreover, Appendix B.3 and Figure 7 show that the improvement of COIN over a language-conditioned baseline is correlated with the similarity of a task's text prompt to its nearest neighboring prompts. Together, these results suggest that demonstration-inferred concepts help the policy select among visually and linguistically similar manipulation behaviors.

Comparison across few-shot adaptation paradigms. In Table 3, we also compare COIN with different ways a policy can be informed about a new task: a task label, a language instruction, a meta-imitation embedding of the demonstrations (TecNets-style), the demonstrations themselves as context (in-context), and a large pretrained VLA (π_0). Every method except π_0 shares COIN's architecture, and all are fine-tuned on the same number of demonstrations. COIN outperforms every alternative at both budgets. Methods trained to use demonstrations do not automatically adapt well: the in-context policy, which attends to the raw demonstration frames, trails COIN, and the TecNets-style embedding, trained with its own metric objective, adapts worse than language. Inferring the task without fine-tuning is not enough either: optimizing a task latent against the frozen policy, as FTL-IGM does, recovers less than half of what fine-tuning achieves and adds little on top of it (Appendix B.4). Retrieving related data from the pretraining set, as STRAP [26] does, complements demonstration conditioning but does not replace it (Appendix E.2). Together, these comparisons indicate that COIN's few-shot advantage comes from a task representation trained for control.

5.3 Improvements Come from Adapting the Demonstration Encoder

To understand what changes during few-shot fine-tuning, we compare full fine-tuning to variants that freeze different parts of COIN. Figure 5b shows that freezing the denoising backbone performs similarly to full fine-tuning across data budgets. In contrast, freezing the demonstration encoder hurts performance, especially in the low-data regime. Figure 8 corroborates this by showing that the denoiser undergoes substantially lighter updates during fine-tuning.

This pattern suggests that pretraining COIN on many concept-action pairs gives the denoiser a robust prior over how different task concepts should be enacted. Few-shot adaptation does not need to relearn the entire action-generation model. This highlights that the action prior largely transfers, while the demonstration-to-concept mapping is what must align to new tasks.

5.4 Robustness to Distribution Shift

We next test whether conditioning on demonstrations makes COIN fragile under distribution shift. LIBERO-Plus [13] perturbs LIBERO's tasks. On this evaluation suite, we apply its robot initial-pose perturbations to 20 LIBERO-90 tasks and evaluate three policies that differ only in their conditioning: demonstrations (COIN), language (trainable T5), and a task label (Table 10 in Appendix E.1). All three lose a similar amount of success, so COIN keeps its advantage from demonstration conditioning: demonstrations are not the fragile part under this shift.

5.5 Latent Size and Number of Demonstrations

Figure 5c varies the concept latent dimension, and Figure 10 in Appendix D.2 the number of demonstrations fed to the encoder. Very small concept latents bottleneck performance, while overly large latents can hurt few-shot fine-tuning, likely because they make it easier to overfit the limited demonstrations. Increasing the number of demonstrations fed to the encoder has a relatively small effect, consistent with our main observation that most of the gain comes from the learned demonstration-conditioning interface rather than averaging many context demonstrations.

6 Discussion and Limitations

COIN shows that demonstrations can serve as more than action supervision: they can provide a task-conditioning signal for inferring reusable behavioral structure. Across controlled domains, COIN is strongest when new tasks require recombining concepts seen during training. On LIBERO, the same mechanism scales to image-based manipulation, improving few-shot fine-tuning and achieving strong full-data performance without language conditioning.

Limitations. First, the learned concept latents are not guaranteed to be human-interpretable, even when the model behaves compositionally. Second, COIN is better at recombining concepts within the training distribution than extrapolating to entirely new concepts, as suggested by the weaker gains on novel-concept tasks. Third, compositional denoising requires one denoiser call per concept latent, increasing inference cost when many concepts are used. Fourth, demonstrations are costlier to collect than an instruction and specific to an embodiment, and a wrong demonstration yields coherent execution of the wrong task. Fifth, one-demonstration results depend on which demonstration is drawn. Finally, we evaluate only in simulation.

Future work should probe the learned concept space more directly, improve the efficiency of compositional denoising, and test whether demonstration-inferred concepts transfer to real robots and longer-horizon task compositions.

Acknowledgments and Disclosure of Funding

The authors thank their mentor, Yilun Du, for his insightful ideas and continued advice. Additionally, we are grateful to our families for their support during the research process, and to our roommates in Quincy around whom we conducted the work leading to this paper. Lastly, the authors thank Chenguang Li, Jorin Overwiening, Haim Sompolinsky, Krishiv Singhal, Aditya Agrawal, and Abhay Malik for their inspiration. Parts of this work were completed while the authors were interns at Waddle Labs, which provided the authors with compute via 8x A100s. This work has been made possible in part by a gift from the Chan Zuckerberg Initiative Foundation to establish the Kempner Institute for the Study of Natural and Artificial Intelligence at Harvard University.

References

[1] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua B. Tenenbaum, Tommi S. Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making? In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=sP1fo2K9DFG. 3

[2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024. URL https://arxiv.org/abs/2410.24164. 1, 3, 8, 15

[3] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. In *Proceedings of Robotics: Science and Systems*, 2023. doi: 10.15607/RSS.2023.XIX.025. URL https://robotics-transformer1.github.io/. 1, 3

[4] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. doi: 10.15607/RSS.2023.XIX.026. URL https://roboticsconference.org/2023/program/papers/026/. 3, 5, 6, 21

[5] Vanya Cohen, Jason Xinyu Liu, Raymond Mooney, Stefanie Tellex, and David Watkins. A survey of robotic language grounding: Tradeoffs between symbols and embeddings. In *International Joint Conference on Artificial Intelligence (IJCAI)*, 2024. 3

[6] Norman Di Palo and Edward Johns. Keypoint action tokens enable in-context imitation learning in robotics. In *Proceedings of Robotics: Science and Systems*, 2024. URL https://arxiv.org/abs/2403.19578. 3

[7] Zibin Dong, Yicheng Liu, Yinchuan Li, Hang Zhao, and Jianye Hao. Conditioning matters: Training diffusion policies is faster than you think. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2505.11123. 1, 2, 4

[8] Danny Driess, Jost Tobias Springenberg, Brian Ichter, Lili Yu, Adrian Li-Bell, Karl Pertsch, Allen Z. Ren, Homer Walke, Quan Vuong, Lucy Xiaoyang Shi, and Sergey Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better. *arXiv preprint arXiv:2505.23705*, 2025. URL https://arxiv.org/abs/2505.23705. 17

[9] Yilun Du. Generalization by Construction. *yilundu.github.io*, August 2026. URL https://yilundu.github.io/perspective/generalization_by_construction.html. 1

[10] Yilun Du, Shuang Li, Yash Sharma, Joshua B. Tenenbaum, and Igor Mordatch. Unsupervised learning of compositional energy concepts. In *Advances in Neural Information Processing Systems*, volume 34, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/838aac83e00e8c5ca0f839c96d6cb3be-Abstract.html. 3, 4

[11] Yilun Du, Conor Durkan, Robin Strudel, Joshua B. Tenenbaum, Sander Dieleman, Rob Fergus, Jascha Sohl-Dickstein, Arnaud Doucet, and Will Sussman Grathwohl. Reduce, reuse, recycle: Compositional generation with energy-based diffusion models and MCMC. In *International Conference on Machine Learning (ICML)*, 2023. 3

[12] Yan Duan, Marcin Andrychowicz, Bradly C. Stadie, Jonathan Ho, Jonas Schneider, Ilya Sutskever, Pieter Abbeel, and Wojciech Zaremba. One-shot imitation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://proceedings.neurips.cc/paper/2017/hash/ba3866600c3540f67c1e9575e213be0a-Abstract.html. 3

[13] Senyu Fei et al. LIBERO-Plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint*, 2025. 10

[14] Chelsea Finn, Tianhe Yu, Tianhao Zhang, Pieter Abbeel, and Sergey Levine. One-shot visual imitation learning via meta-learning. In *Proceedings of the 1st Annual Conference on Robot Learning*, volume 78 of *Proceedings of Machine Learning Research*, pages 357–368. PMLR, 2017. URL https://proceedings.mlr.press/v78/finn17a.html. 3

[15] Vedant Gupta, Haotian Fu, Calvin Luo, Yiding Jiang, and George Konidaris. Learning parameterized skills from demonstrations. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2510.24095. 7

[16] Siddhant Haldar, Zhuoran Peng, and Lerrel Pinto. BAKU: An efficient transformer for multi-task policy learning. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://arxiv.org/abs/2406.07539. 8, 17

[17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. URL https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html. 21

[18] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems*, volume 29, pages 4565–4573, 2016. URL https://proceedings.neurips.cc/paper/2016/hash/cc7e2b878868cbae992d1fb743995d8f-Abstract.html. 2

[19] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html. 3, 5

[20] Stephen James, Michael Bloesch, and Andrew J. Davison. Task-embedded control networks for few-shot imitation learning. In *Conference on Robot Learning (CoRL)*, 2018. 2, 3, 4, 8, 15

[21] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 9902–9915. PMLR, 2022. URL https://proceedings.mlr.press/v162/janner22a.html. 3

[22] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025. 3

[23] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P. Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 2025. URL https://proceedings.mlr.press/v270/kim25c.html. 1, 3

[24] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, volume 36, 2023. URL https://arxiv.org/abs/2306.03310. 7

[25] Nan Liu, Shuang Li, Yilun Du, Antonio Torralba, and Joshua B. Tenenbaum. Compositional visual generation with composable diffusion models. In *European Conference on Computer Vision*, pages 423–439. Springer, 2022. URL https://arxiv.org/abs/2206.01714. 3, 4, 5

[26] Marius Memmel, Jacob Berg, Bingqing Chen, Abhishek Gupta, and Jonathan Francis. STRAP: Robot sub-trajectory retrieval for augmented policy learning. In *International Conference on Learning Representations (ICLR)*, 2025. 9, 20

[27] Atharva Mete, Haotian Xue, Albert Wilcox, Yongxin Chen, and Animesh Garg. QueST: Self-supervised skill abstractions for learning continuous control. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://arxiv.org/abs/2407.15840. 17

[28] Aviv Netanyahu, Yilun Du, Antonia Bronars, Jyothish Pari, Joshua B. Tenenbaum, Tianmin Shu, and Pulkit Agrawal. Few-shot task learning through inverse generative modeling. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://arxiv.org/abs/2411.04987. 3, 4, 6, 15, 20

[29] Andrew Y. Ng and Stuart J. Russell. Algorithms for inverse reinforcement learning. In *Proceedings of the Seventeenth International Conference on Machine Learning*, pages 663–670. Morgan Kaufmann, 2000. URL https://ai.stanford.edu/~ang/papers/icml00-irl.pdf. 2

[30] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Pannag Sanketi, Quan Vuong, Ted Xiao, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, 2024. doi: 10.15607/RSS.2024.XX.090. URL https://octo-models.github.io/. 1, 3

[31] Physical Intelligence, Kevin Black, Noah Brown, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025. 3

[32] Dean A. Pomerleau. Alvinn: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, volume 1, 1988. URL https://proceedings.neurips.cc/paper/1988/hash/812b4ba287f5ee0bc9d43bbf5bbe87fb-Abstract.html. 1, 2

[33] Moritz Reuss, Jyothish Pari, Pulkit Agrawal, and Rudolf Lioutikov. Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning. *arXiv preprint arXiv:2412.12953*, 2024. URL https://arxiv.org/abs/2412.12953. 3, 8, 15, 17

[34] Moritz Reuss, Hongyi Zhou, Marcel Rühle, Ömer Erdinç Yağmurlu, Fabian Otto, and Rudolf Lioutikov. FLOWER: Democratizing generalist robot policies with efficient vision-language-action flow policies. In *Proceedings of the 9th Conference on Robot Learning*, 2025. URL https://arxiv.org/abs/2509.04996. 8, 17

[35] Jocelin Su, Nan Liu, Yanbo Wang, Joshua B. Tenenbaum, and Yilun Du. Compositional image decomposition with diffusion models. In *International Conference on Machine Learning (ICML)*, 2024. 3

[36] Vitalis Vosylius and Edward Johns. Instant policy: In-context imitation learning via graph diffusion. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=je3GZissZc. 3

[37] Lirui Wang, Jialiang Zhao, Yilun Du, Edward H. Adelson, and Russ Tedrake. Poco: Policy composition from and for heterogeneous robot learning. In *Proceedings of Robotics: Science and Systems*, 2024. doi: 10.15607/RSS.2024.XX.127. URL https://arxiv.org/abs/2402.02511. 3, 5

[38] Yanwei Wang, Lirui Wang, Yilun Du, Balakumar Sundaralingam, Xuning Yang, Yu-Wei Chao, Claudia Perez-D'Arpino, Dieter Fox, and Julie A. Shah. Inference-time policy steering through human interactions. In *IEEE International Conference on Robotics and Automation*, 2025. URL https://arxiv.org/abs/2411.16627. 3

[39] Yuxin Wu and Kaiming He. Group normalization. In *Proceedings of the European Conference on Computer Vision*, pages 3–19, 2018. URL https://openaccess.thecvf.com/content_ECCV_2018/html/Yuxin_Wu_Group_Normalization_ECCV_2018_paper.html. 21

[40] Mengdi Xu, Yikang Shen, Shun Zhang, Yuchen Lu, Ding Zhao, Joshua B. Tenenbaum, and Chuang Gan. Prompting decision transformer for few-shot policy generalization. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 24631–24645. PMLR, 2022. URL https://proceedings.mlr.press/v162/xu22g.html. 3

[41] Xinyu Zhang and Abdeslam Boularias. One-shot imitation learning with invariance matching for robotic manipulation. In *Proceedings of Robotics: Science and Systems*, 2024. URL https://arxiv.org/abs/2405.13178. 3

[42] Ruijie Zheng, Ching-An Cheng, Hal Daumé III, Furong Huang, and Andrey Kolobov. Prise: Llm-style sequence compression for learning temporal action abstractions in control. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2402.10450. 7

[43] Allan Zhou, Eric Jang, Daniel Kappler, Alexander Herzog, Mohi Khansari, Paul Wohlhart, Yunfei Bai, Mrinal Kalakrishnan, Sergey Levine, and Chelsea Finn. Watch, try, learn: Meta-learning from demonstrations and reward. In *International Conference on Learning Representations*, 2020. URL https://arxiv.org/abs/1906.03352. 3

[44] Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence*, pages 1433–1438, 2008. URL https://www.aaai.org/Papers/AAAI/2008/AAAI08-227.pdf. 2

[45] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html. 1, 3

A Composition experiments

A.1 Isolating the composition rule in object rearrangement

Section 4.2 compares rules for combining two trained relations on held-out compositions. This section gives the setup and the per-composition results.

Models. For this experiment we trained two models on a regenerated dataset of 18,000 single-relation states (1,500 for each of the 12 training relations), each for 10^5 steps with batch size 64, an encoder of width 128, and a denoiser of width 1024: a $K{=}2$ model with latents of size $L{=}32$, and a $K{=}1$ model with the same total latent size ($L{=}64$). Both generate states that satisfy the trained relations in at least 99.9% of cases, so capacity does not limit either of them.

Rules. Each held-out composition combines two trained relations. We encode one demonstration of each relation and compare (i) averaging the two $K{=}1$ latents and denoising once; (ii) denoising with each $K{=}1$ latent and averaging the two noise predictions; and (iii) encoding each demonstration with the $K{=}2$ model, taking from each the latent that carries its relation, and summing the two noise predictions. Every cell uses 1000 generated states.

Held-out composition	(i)	(ii)	(iii)
circle right of triangle ∧ square above triangle	0.34	0.70	0.99
triangle right of square ∧ circle above square	0.30	0.71	0.99
square right of circle ∧ triangle above circle	0.36	0.70	0.96
Mean	0.33	0.70	0.98

Table 4: **Composition rules on the three held-out compositions** of the object-rearrangement domain (1000 generated states per cell): (i) average the latents, (ii) average the noise predictions, (iii) one latent per concept. Conditioning on either relation alone succeeds in 0.17–0.22 of cases.

Two further baselines support the result. Encoding demonstrations *of the composition itself* with a $K{=}1$ model—information COIN never receives—reaches only 0.43–0.52, and single-latent encoders trained with a metric-learning or variational objective reach at most 0.76 under any of the rules available to them. Combining noise predictions is therefore what lets two separately inferred relations combine, and keeping each relation in its own latent is what lifts composition to near-perfect success.

A.2 Task latents on LIBERO

What the task latent encodes. On the full-data checkpoint ($K{=}1$, $L{=}128$, 95.1% on LIBERO-90), 21 of 22 task-factor values (verbs, objects, receptacles, and spatial relations) are linearly decodable from the task latent on held-out tasks. A single demonstration identifies its task among all 90 in every one of 4500 trials, while the initial object poses of the demonstration are not decodable ($R^2 = -0.05$): the latent describes the task, not the episode.

Latent arithmetic. We use a policy trained on LIBERO-90 with ten tasks held out, each of which combines an object and a receptacle that appear in training but never together (Appendix A.3). For "put the butter in the basket", we add the latents of butter-in-tray and sauce-in-basket, subtract the latent of sauce-in-tray, and condition the frozen policy on the result. It succeeds in 20 of 30 rollouts, whereas conditioning on the held-out task's own demonstrations succeeds in none of 30.

Latent editing. Each edit moves a task's placement from location A to location B. The edit vector is the average of $z_Q - z_P$ over other training-task pairs (P, Q) in the same scene that differ only in placing at B instead of A; the edited task and its target are never among these pairs, and the conditioning demonstrations never show the edited goal. Over five edits (10 rollouts each), the edited latent achieves the new placement in 16 of 50 rollouts, against 3 of 50 unedited and 5 of 50 for random vectors of the same norm. For example, an edit turns "put the bowl on the plate" into putting the bowl on top of the cabinet, a goal its conditioning never demonstrated.

A.3 Few-shot adaptation on a compositional split

LIBERO's standard held-out tasks (indices 80–89) cluster in a few scenes. We therefore also built a compositional split: ten LIBERO-90 tasks, spread over nine scenes, whose object–receptacle combination never appears in the remaining 80 training tasks while each individual object, receptacle, relation, verb, and scene does. Zero-shot success on these tasks is about 7%. Under the few-shot

protocol of Appendix D, one demonstration per task lifts COIN to 48%, close to the 52.3% it reaches on the standard held-out tasks, and five demonstrations reach 58%.

B LIBERO: baselines and ablations

B.1 Baselines and parameter counts

Every baseline we trained shares COIN's policy: the transformer denoiser and per-view ResNet visual encoders of Appendix F.3, trained from random initialization with the same data processing and recipe. Only the conditioning pathway differs (Table 5).

- **Task label** looks up a learned embedding of the ground-truth task index and projects it into the place of COIN's task latent.
- **Language** embeds the instruction with T5-base and projects it into the same place. The frozen variant computes the embedding once with a frozen encoder; the trainable variant fine-tunes the T5 encoder end-to-end with the policy.
- **TecNets-style** uses COIN's demonstration encoder with the TecNets recipe [20]: an L2-normalized 128-dimensional embedding trained with a hinge-margin metric loss, averaged over demonstrations.
- **Variational** uses COIN's encoder with a Gaussian output, adds a KL penalty (weight 10^{-4}) to the denoising loss, samples during training, and uses the mean at evaluation.
- **In-context** has no demonstration encoder: the policy's observation encoder embeds a strided subset of the demonstration frames, and the denoiser attends to them as conditioning tokens.
- **Concatenation** encodes $K{=}2$ latents and concatenates them into one denoiser input instead of summing two noise predictions.
- **Test-time optimization** follows FTL-IGM [28]: it optimizes a task latent against the frozen pretrained policy's denoising loss on the k demonstrations for up to 1500 gradient steps, initialized either randomly or at the encoder's output, and we report the best of a grid of three learning rates and four step counts.

We also fine-tune π_0 [2] ourselves with its official openpi recipe (Appendix G). MoDE [33] is a language-conditioned diffusion policy whose denoiser is a mixture of expert transformers routed by noise level; it is the strongest diffusion policy reported on LIBERO-90 trained from scratch, and we quote its published results.

Model	Conditioning	Cond. module	Total
COIN, ResNet-50 (full data)	demonstrations → MLP encoder → task latent	11.6M	471.0M
COIN, ResNet-18 (few-shot base)	same	11.5M	445.8M
Language, frozen T5 (ResNet-50)	frozen T5-base embedding + linear projection	0.3M	459.7M
Language, trainable T5 (ResNet-18)	T5-base encoder fine-tuned end-to-end	134.3M	568.8M
Task label (ResNet-18)	90-entry embedding table + projection	0.1M	434.4M
In-context (ResNet-18)	demonstration frames as conditioning tokens	0.03M	434.4M
Variational (ResNet-18)	COIN's encoder with a Gaussian output and KL penalty	11.6M	445.9M
TecNets-style (ResNet-18)	L2-normalized embedding, metric loss, averaged	1.7M	436.1M
Concatenation (ResNet-18)	two latents concatenated into one denoiser input	11.5M	445.8M
π_0 (our fine-tune)	language and vision, flow-matching action expert	—	3.3B
MoDE†	language, mixture-of-experts diffusion transformer	—	277–460M
OpenVLA†	language and vision, autoregressive action tokens	—	7B

Table 5: **Baseline architectures and parameter counts.** Rows above the rule share COIN's denoiser and visual encoders; only the conditioning pathway differs. "Total" is the exact parameter count of the trained checkpoint. The frozen T5-base encoder of the frozen-language row (134.3M) is applied once to the 90 instructions and is not part of the trained policy. †Quoted from the original publications.

B.2 Number of concept latents

All LIBERO results use $K{=}1$. At matched latent capacity, $K{=}2$ ($L{=}64$) performs on par with $K{=}1$ ($L{=}128$) on LIBERO-90: 94.4 vs. 95.1, and the sign of the gap flips under a second training seed. With more than one latent on LIBERO, the latents learn the same function.

LIBERO's own task definitions explain why there is little to compose. Table 6 counts the goal conditions of every task with the benchmark's goal parser: 79 of the 90 LIBERO-90 tasks have a single goal condition, and each of the remaining 11 is trained with its own 50 demonstrations.

Suite	Tasks	1 goal condition	2 goal conditions	3 goal conditions
LIBERO-90	90	79	11	0
LIBERO-Long	10	1	8	1
LIBERO-Goal, -Spatial, -Object	30	30	0	0

Table 6: **Goal conditions per task in the LIBERO suites**, counted with the benchmark's goal parser over the shipped task definitions (one condition is one predicate in the task's goal conjunction).

B.3 Shared instructions

Many LIBERO-90 instructions recur across scenes with different layouts (Figure 6). Grouping tasks by how many other tasks share their exact instruction (62, 16, and 12 tasks share it with 0, 1, and 2 or more others), COIN reaches 96, 94, and 97%; π_0 reaches 96, 98, and 79%; and the language-conditioned Diffusion Policy reaches 70, 66, and 52% (Figure 2b). π_0 and the language policy do worst on the most shared instructions, while COIN does not degrade. Across tasks, COIN's advantage over the language policy grows with how similar a task's instruction is to its nearest neighbors (Figure 7). Instruction sharing does not explain the whole gap, however: a task-label policy, which identifies every task perfectly, does no better than the language policy (66.2 vs. 64.5).



Figure 6: **Same instruction, different scenes.** "Put the black bowl on top of the cabinet" appears in several LIBERO kitchens with different layouts; a demonstration shows how the task is done in a given scene.

B.4 Test-time optimization

We optimize a task latent against the frozen LIBERO-80 policy on the 10 held-out tasks, without fine-tuning. From one and five demonstrations, the best optimization reaches 22% and 31%, after about 10 to 19 minutes per task on an A100 GPU; the encoder's output alone reaches 3% and 7%. Fine-tuning COIN on the same demonstrations reaches 52.3% and 77.5%, and optimizing the latent on top of the fine-tuned model changes success by at most 2.5 points.

C Full-data LIBERO-90

Main result. COIN with a ResNet-50 visual encoder reaches 96.4, 96.6, and 96.7% on LIBERO-90 over three training seeds (96.6 ± 0.2; 90 tasks $\times$ 20 rollouts, DDIM-10, fixed epoch 19). The 95% confidence interval of a single evaluation is $[94.7, 97.9]$.

Sampler. Sampler choice shifts absolute success (DDPM with 100 steps scores 89.0 against 95.1 for DDIM with 10 steps on the same checkpoint), so every comparison in this paper holds the sampler fixed.

The π_0 comparison. π_0 has no published LIBERO-90 result. We fine-tuned it on LIBERO-80 with the official recipe (Appendix G). Evaluated under our protocol on the 80 tasks, it reaches 92.3%, against 92.6% for COIN trained on the same 80 tasks: a tie.

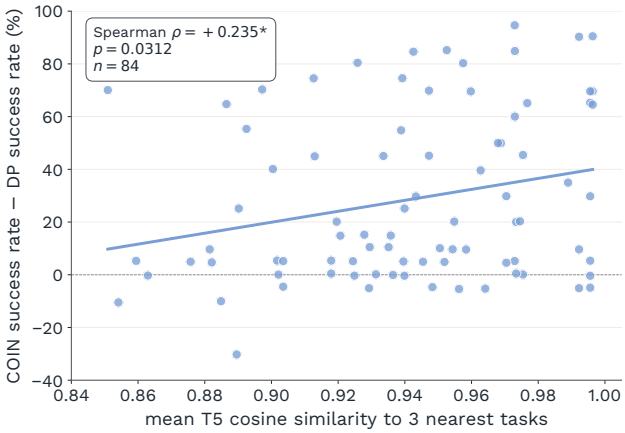


Figure 7: **COIN helps more when instructions have similar neighbors.** Each point is a LIBERO-90 task: the x-axis is the mean T5 cosine similarity between its instruction and its three nearest neighbors; the y-axis is COIN's success minus the language policy's. The plot covers the 84 tasks whose similarity exceeds 0.85. The correlation is weak but significant and not causal.

Published results. Table 7 collects the published LIBERO-90 results we are aware of. The quoted entries follow their own protocols. COIN exceeds every published result that does not use robot-data pretraining, while itself training from random initialization.

Method	LIBERO-90	Params	Pretraining	Protocol (as stated)
$\pi_{0.5}$-KI† [8]	96.0	∼3B	vision-language + robot data	not stated
MoDE† [33]	95.0	277–460M	∼200k robot trajectories	several seeds
FLOWER† [34]	94.7 ± 1.0	947M	vision-language + robot data	50 trials, 3 seeds
$\pi_{0.5}$-KI, no robot data† [8]	92.7	∼3B	vision-language	not stated
MoDE, from scratch† [33]	91.0	277M	none	several seeds
BAKU† [16]	90.0	—	none	paper protocol
QueST† [27]	88.6 ± 0.4	∼20M	none	40 rollouts, 4 seeds
COIN (ResNet-50)	96.6 ± 0.2	0.47B	none (random initialization)	20 rollouts, 3 seeds

Table 7: **Published LIBERO-90 results.** †Quoted from the original publications under their own protocols.

D Few-shot protocol and results

Protocol. We pretrain COIN (K=1, L=128, ResNet-18) on LIBERO-80 (task indices 0–79) with the full-data recipe and use epoch 19 as the base for every held-out task (indices 80–89). Each method is fine-tuned on k demonstrations per held-out task with a cosine schedule over 1000 epochs, so the learning rate stays within 3% of its peak of 10^{-4} through the evaluated epochs; we evaluate with 20 rollouts per task at two fixed epochs (49 and 99) and report the mean of the two. The demonstrations are drawn with a recorded seed; COIN, the trainable-language policy, and the TecNets-style policy share draws at one and five demonstrations where stated below. COIN's encoder conditions only on the run's own k demonstrations; with one demonstration it fills both context slots.

Demonstrations per task	1	5	10	25	50
COIN	52.3 ± 4.4	77.5 ± 0.3	78.1 ± 0.1	83.1 ± 1.6	84.5
Language (trainable T5)	39.8	67.2 ± 0.4	68.5 ± 1.8	81.4 ± 1.4	77.5
π_0, fine-tuned	36.5	59.5	72.0	77.5	86.5

Table 8: **Few-shot success across budgets** on the 10 held-out tasks. ±: spread over demonstration draws.

Comparison with π_0. COIN leads at every budget up to 25 demonstrations. Extending the one- and five-demonstration evaluations to 50 rollouts per task gives 52.7 vs. 32.2 and 75.5 vs. 55.4 (both $p = 0.0098$).

Comparison with the language policy. At five and ten demonstrations both methods use the same two demonstration draws. COIN leads by 10.4 points at five ($p \approx 0.001$) and by 9.6 at ten ($p \approx 0.03$); the two are tied at 25.

Other few-shot baselines. Without fine-tuning, both the TecNets-style encoder and COIN's encoder are near zero on these tasks. The in-context policy reaches 38.5 at one demonstration and 70.5 at five, and the task-label policy 25.3 and 65.5.

Demonstration draws. With one demonstration, which demonstration is drawn moves COIN's success by several points, so we average over draws where we have them.

D.1 What fine-tuning changes

Figure 8 measures where fine-tuning acts. After fine-tuning on five demonstrations per held-out task, we give the pretrained and the fine-tuned model the same inputs and measure how far each module's output moves. The encoder's task latents move far more than the denoiser's noise predictions, especially on held-out tasks: fine-tuning mostly re-maps the new tasks' demonstrations to new latents and leaves the denoiser, which turns a latent into actions, nearly unchanged. Figure 5b shows the same from the other side: freezing the denoiser performs comparably to full fine-tuning, whereas freezing the demonstration encoder hurts, most of all with few demonstrations.

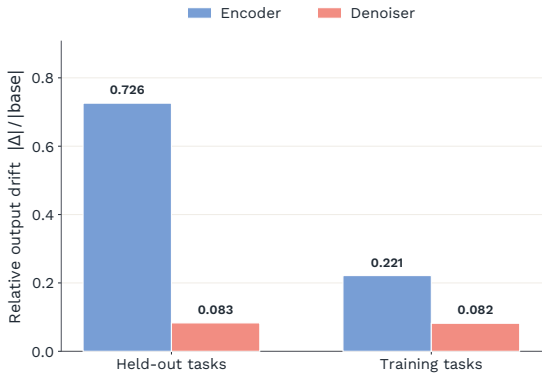


Figure 8: **Fine-tuning mainly changes the encoder.** Each bar is the relative change of a module's output on fixed inputs, $\|f_{\text{ft}}(x) - f_{\text{base}}(x)\| / \|f_{\text{base}}(x)\|$, between the pretrained model and the model fine-tuned on five demonstrations per held-out task. For the encoder, x is a demonstration and the output its task latent; for the denoiser, x is a fixed latent, observation, diffusion step, and noisy action, and the output its noise prediction. Inputs come from held-out tasks (left) or training tasks (right).

D.2 Backbone, regularization, and encoder hyperparameters

A larger visual backbone does not help few-shot adaptation. Figure 9 runs the same few-shot protocol with COIN's per-camera visual encoder set to ResNet-18 or ResNet-50: the smaller backbone adapts better from both one and five demonstrations, so all few-shot experiments use ResNet-18, while the full-data results use ResNet-50. Mild dropout on image features leaves full-data success unchanged, while stronger dropout hurts.

E Robustness and the cost of demonstrations

Wrong demonstrations. We use the 28 LIBERO-90 tasks whose instruction is shared with at least one other task. Conditioning only on demonstrations of the task with the same instruction in another scene drops success from 96.4% to 26.8%; conditioning only on demonstrations of a different task in the same scene drops it from 93.3% to 0.0% (12 tasks with such a partner). The failures are coherent executions of the other task (Figure 11). When only one of the two context demonstrations is wrong, success drops from 96.3% to 53.2% (same instruction, other scene) and 28.9% (same scene, other task).

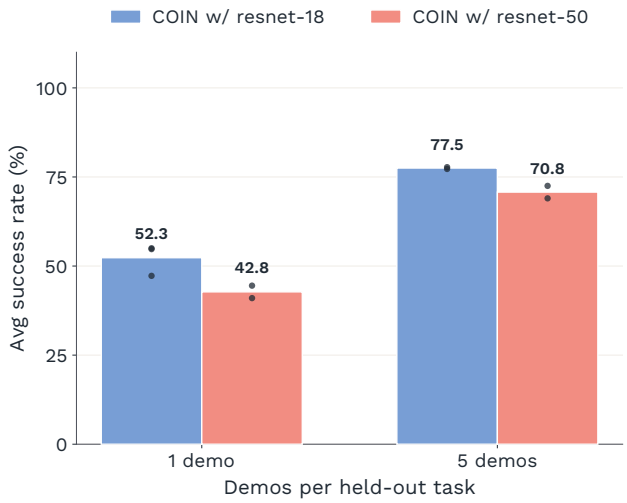


Figure 9: **Visual backbone in few-shot fine-tuning.** Success of COIN with a ResNet-18 or a ResNet-50 visual encoder, pretrained on LIBERO-80 and fine-tuned on one or five demonstrations per held-out task. Bars are means over demonstration draws; dots are the individual draws.

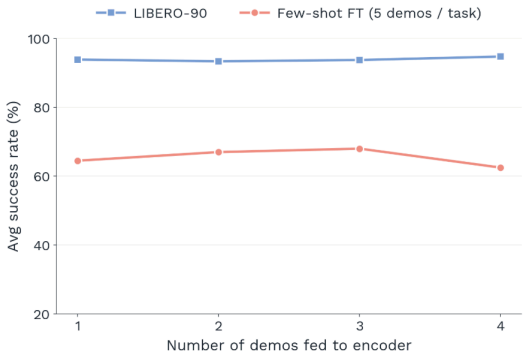


Figure 10: **Number of context demonstrations.** Increasing the number of demonstrations fed to the encoder has little effect on either full-data or few-shot performance.

Wrong instructions. Language conditioning is equally sensitive. Replacing the instruction with a familiar but wrong one drops the frozen-language policy from 63.4% to 15.9% on the shared-instruction tasks, and the trainable-language policy from 86.3% to 10.4% on the 23 such tasks in its LIBERO-80 training set. Appending scene names to shared instructions does not help either policy (Figure 12).

Degraded demonstrations. We corrupt the correct demonstration on all 90 LIBERO-90 tasks with a fixed context pair per task (Table 9). Truncation keeps the first fraction of every stream of the demonstration (at least 12 frames); frame dropping keeps every second frame across the whole demonstration; pixel noise adds Gaussian noise ($\sigma = 25$ on 8-bit images) to both cameras in every frame.

Context demonstration	Success
clean	95.1
first 75% of frames	77.7
first 50% of frames	37.4
first 25% of frames	21.3
every second frame	59.9

Table 9: **Degraded demonstrations** on LIBERO-90.

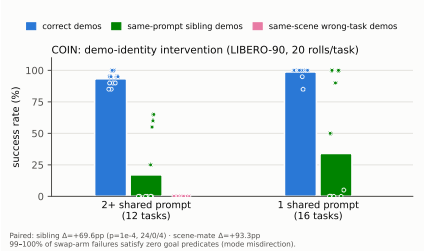


Figure 11: **Wrong demonstrations.** Correct (C), same-instruction other-scene (S), and same-scene other-task (X) demonstrations; bars are task means with per-task points.

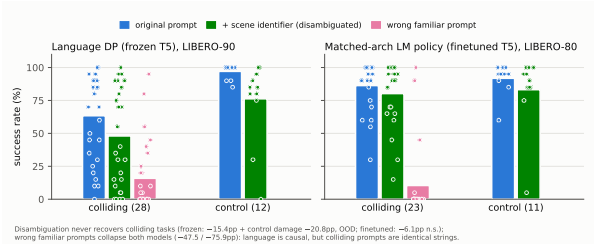


Figure 12: **Wrong instructions.** Original (O), scene-disambiguated (D), and wrong but familiar (W) instructions, for the frozen-language policy and the trainable-language policy.

E.1 LIBERO-Plus

Conditioning	Clean	Perturbed
language (trainable T5)	85.0	51.8
task label	80.5	50.0
COIN	**90.8**	**60.5**

Table 10: **LIBERO-Plus.** Success on 20 LIBERO-90 tasks with and without robot initial-pose perturbations.

E.2 STRAP retrieval

We implement STRAP [26] as published except for segmentation. Each target demonstration is split at every opening or closing of the gripper (segments shorter than 20 frames are merged into a neighbor), instead of with UVD. Each segment is embedded with DINOv2-base features of the third-person camera and matched against the LIBERO-80 pretraining demonstrations with subsequence dynamic time warping (cosine distance), and the 100 best-matching sub-trajectories per target task are added to the fine-tuning data; retrieved segments keep the identity of their source task. With one target demonstration, adding retrieved data gives COIN 58.5%, below the 71.5% of fine-tuning on the target demonstration alone for the same number of batches; with five, it makes no significant difference. On the same retrieved data, the language-conditioned policy reaches only 6.0% with one target demonstration.

F Domain and training details

F.1 Object rearrangement

Each scene contains three objects (a circle, a triangle, and a square) with continuous positions, and the policy generates all object positions in one step. Training data and splits follow Netanyahu et al. [28]: the 12 training concepts are “A right of B” and “A above B” for ordered pairs of objects, and the held-out tasks are five compositions of two training relations and five novel concepts (four diagonal relations and the three objects equally spaced on a circle), each given by five demonstrations. The encoder is a two-layer MLP (hidden width 128) feeding a shared head (hidden width 1024) that emits K latents of size $L{=}32$; the denoiser is an MLP. Accuracy is the fraction of generated states that satisfy the task’s predicates, and a composition counts only if both relations hold in the same state. Table 11 lists the hyperparameters.

F.2 Goal-oriented navigation

We use the AGENT navigation environment of Netanyahu et al. [28] with its training, composition, and new-initial-state splits. An agent navigates to one of two objects, a target and a distractor, each with one of four colors and one of four shapes. The four training concepts name one attribute of the target (a red or yellow object, a bowl or a cube); the five held-out tasks bind two attributes (red bowl, yellow bowl, red cube, yellow cube, purple cone) and are each given by five demonstrations,

Table 11: Object rearrangement hyperparameters.

Encoder hidden width / head hidden width	128 / 1024
Number of concept latents K / latent size L	2 / 32
Denoiser	MLP, single step
Diffusion steps	100, cosine β schedule, ϵ-prediction
Loss	ℓ_2 on ϵ, action weight 10
Batch size	32, gradient accumulation 2 (effective 64)
Optimizer / learning rate	Adam, 2×10^{-4}
EMA decay / training steps	0.995 / 2×10^5
Sampler	DDPM, 100 steps; x_0 clipped to $[-1, 1]$

with a distractor that shares the target's color or shape. The encoder is a three-layer MLP (hidden width 1024) over the demonstration, pooled at stride 10. The denoiser is a temporal U-Net with four resolutions (base width 128, channel multipliers $(1, 2, 4, 8)$, kernel size 5) over action chunks of length 128. In closed loop, the agent moves toward the planned state five steps ahead and replans from each new observation; an episode succeeds if the agent reaches the target before the distractor (Table 12).

Table 12: Goal-oriented navigation hyperparameters.

Encoder layers / hidden width / pooling stride	3 / 1024 / 10
Number of concept latents K / latent size L	2 / 128
Denoiser base width / channel multipliers / kernel	128 / $(1, 2, 4, 8)$ / 5
Action horizon	128
Diffusion steps	100, cosine β schedule, ϵ-prediction
Loss	ℓ_2 on ϵ, action weight 10
Batch size	32, gradient accumulation 2 (effective 64)
Optimizer / learning rate	Adam, 2×10^{-4}
EMA decay / training steps	0.995 / 10^6
Sampler	DDPM, 100 steps; x_0 clipped to $[-1, 1]$

F.3 LIBERO architecture and training

Visual and demonstration encoders. Each camera view has its own ResNet-18 or ResNet-50 [17], with batch normalization replaced by group normalization [39], trained from random initialization (no ImageNet weights). Images are randomly cropped to 116×116 in training and center-cropped at evaluation; pooled features of each view are projected to a 128-dimensional token, and tokens from two consecutive steps form the observation. The same observation encoder embeds the frames of the context demonstration; every tenth frame's features are flattened and passed through a three-layer MLP (hidden width 1024) that outputs the task latent.

Denoiser. The denoiser is the transformer of Diffusion Policy [4] with 24 layers, 16 heads, and width 1024 (attention dropout 0.3, residual dropout 0.1), with a causal mask over the action sequence and a single MLP projecting the conditioning (Table 13). We do not use classifier-free guidance.

Table 13: LIBERO hyperparameters (identical for every suite).

Demonstration encoder	MLP (3 layers, width 1024), every tenth frame
Concept latents K / latent size L	1 / 256 (full data), 128 (few-shot)
Observation steps / action horizon / actions executed	2 / 16 / 8
Denoiser layers / heads / width	24 / 16 / 1024
Diffusion steps	100, squared-cosine schedule, ϵ-prediction
Batch size	64×8 accumulation (effective 512)
Optimizer / weight decay	AdamW (0.9, 0.999) / 0.05
Learning rate	10^{-4}, cosine schedule
Warmup steps	1000 (ResNet-18), 100 (ResNet-50, fine-tuning)
Schedule length	60 epochs (ResNet-50), 200 (ResNet-18)
Evaluated epoch (full data)	19
EMA	power schedule, exponent 0.75
Sampler	DDIM, 10 steps
Evaluation	20 rollouts per task, at most 520 steps

G Code, compute, and statistics

Code. Code for training, evaluation, and every figure, together with configuration files for all experiments, will be available at https://github.com/DozenDucc/compositional-concept-inference.

Compute. Each COIN and baseline training run used one NVIDIA A100 40GB GPU. Inference with DDIM-10 takes about 430 ms per chunk of 8 actions on an A100. Test-time optimization ran on A100 GPUs (Appendix B.4). We fine-tuned π_0 with the official openpi code: full fine-tuning of the π_0 base checkpoint on LIBERO-80 for 20 epochs (375k steps, batch size 32, four GPUs, cosine learning rate with peak 2.5×10^{-5}, EMA 0.99), followed by 30k steps of held-out fine-tuning from the retained checkpoint (learning rate 10^{-5}; at 50 demonstrations we also ran a LoRA variant and report the better of the two).

Statistics. Unless stated otherwise, LIBERO results are averages over tasks with 20 rollouts per task and DDIM-10. Confidence intervals are 95% cluster bootstraps over tasks and rollouts, and per-task comparisons use paired Wilcoxon or sign-flip permutation tests.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state two scoped claims, each evaluated in the paper: composing inferred concept latents solves held-out combinations of concepts in object rearrangement and navigation (Section 4); and on LIBERO, where extra latents bring no gain and $K{=}1$ is used, the gains come from a task representation trained through the policy's loss (Section 5). The LIBERO-90 result is compared with published results and with π_0 evaluated under our protocol.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 6 states the limitations: composition gains require tasks that combine concepts, extrapolation to novel concepts is weak, demonstrations are costlier than instructions and a wrong demonstration produces execution of the wrong task, one-demonstration results depend on the drawn demonstration, and all evaluation is in simulation. Appendix E measures the costs of wrong and degraded demonstrations.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best

judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present formal theoretical results, theorems, or proofs. The mathematical expressions in Section 3 define the model and training objective rather than stating theoretical guarantees.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 3 and Appendices B–F describe the model, the baselines, every evaluation protocol, and all hyperparameters; Appendix D gives the few-shot protocol in full.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.

(c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code for training, evaluation, and every figure, with configuration files for all experiments, will be released at https://github.com/DozenDucc/compositional-concept-inference (Appendix G). All datasets used are public.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Sections 4 and 5 describe the settings, baselines, splits, and protocols; Appendix F gives the architecture, optimizer, diffusion, batch-size, and training details for each domain, and Appendix D the few-shot protocol.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The LIBERO-90 headline reports the mean and standard deviation over three training seeds and a 95% cluster-bootstrap interval; the encoder-objective ablation uses two training seeds; one-demonstration few-shot results report the standard deviation over three demonstration draws. Key comparisons carry paired per-task tests and confidence intervals, and evaluation noise is quantified (Appendix G). Other cells are single training runs and are labeled as such.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix G reports the hardware (one NVIDIA A100 40GB GPU per training run; A100 GPUs for test-time optimization), inference cost, and the π_0 fine-tuning recipe.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics https://neurips.cc/public/EthicsGuidelines?

   Answer: [Yes]

   Justification: We reviewed the NeurIPS Code of Ethics and believe the work conforms to it.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.

- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

    Justification: This is foundational research on learning from demonstrations, evaluated in simulation; it does not release a deployable system. As with other robot-learning methods, any real-world deployment would require safety validation, which this work does not address.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper does not release a high-risk model such as a pretrained language model, image generator, scraped dataset, or deployed robot-control system.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: All assets are public and credited: LIBERO (MIT), LIBERO-Plus (used as released), Diffusion Policy (MIT), openpi and the π_0 checkpoints (Apache-2.0), T5 (Apache-2.0), DINOv2 (Apache-2.0), and the AGENT environment and FTL-IGM code, cited in Section 4.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We release our code with documentation (Appendix G).

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs are not a component of the method. We used LLM-based assistants for editing, for understanding technical concepts, for implementing standard methods, for running experiments, and for visualizing results, as declared on OpenReview; the authors verified all results.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.